Methodology Companion

An Empirical Analysis of Transfer for Dynamic Path Planning

2026-07-23 — anonymized companion document

This document collects, in one place and at full protocol detail, the methodology of the accompanying submission: environments, learned heuristic providers, planner integrations, transfer and adaptation protocols, the bounded-observation study, evaluation metrics, statistical procedures, and the methodological safeguards under which every claim was produced. Results appear only where a number justifies a design choice.

1 Overview and Design Logic

The study asks whether heuristics learned on training maps transfer—zero-shot or from a few labeled target maps—to new environments for dynamic path planning, and what governs the answer. Two coupled experimental programs address it: a *discrete* grid-world program and a *continuous* roadmap program, spanning thirteen preregistered stages (C1–C13) plus discrete counterparts.

Two design principles organize everything. First, *static suites serve as a proving ground*: supervision there is exact and confounds are controllable, so choices of training objective and planner integration are validated on static maps before being evaluated on the dynamic target, where the classical baseline is weakest. Second, *every comparison isolates one factor*: twin models differing in a single input, integration comparisons reusing identical frozen weights, and architecture arms trained under one matched recipe. Transfer results are otherwise confounded by the objective, the integration, and the supervision content of a "shot"; the methodology below exists to remove those confounds.

2 Environments and Substrates

2.1 Discrete grid worlds

Maps are occupancy grids from 20×20 up to 256×256 cells with static obstacles and, in dynamic variants, moving obstacles; the planner is A* (Hart et al., 1968) with the Manhattan-distance baseline heuristic. Evaluation suites cover in-distribution families, structural out-of-distribution families, and scale out-of-distribution sizes (e.g., models trained at smaller scales evaluated at 128×128–256×256), under fixed expansion budgets with 100 episodes per suite/budget row in the controlled runs (22 suites × 3 budgets in the canonical transfer grid).

2.2 Continuous roadmaps

A point robot plans in 2-D environments with procedurally generated obstacles via a probabilistic roadmap (PRM; Kavraki et al., 1996) of ∼192 nodes with k=7 connectivity; the baseline heuristic

is Euclidean distance. Obstacle families include hard maze, dense maze, rooms, spiral, bugtrap, and large-room layouts; families are deliberately held out of training to measure family-level transfer.

2.3 Dynamic environments

Dynamic obstacles follow deterministic trajectories. The search state is a (node, time) pair, following space–time A* (Silver, 2005; Phillips and Likhachev, 2011): waiting and timing are part of the plan, and the search expands states of a time-expanded graph. Six dynamic suites (crossing, maze, dense maze, rooms, large rooms, spiral) use per-suite budgets; the heavy configuration trains on 53 usable maps and evaluates on 20 held-out maps per suite.

2.4 Calibration and binding budgets

Every suite carries a per-suite *binding budget* of node expansions; success means reaching the goal within it. Budgets are calibrated so the classical baseline lands in a discriminative band (observed 0.25–0.62 success across formal static cells; e.g., binding budgets of 140 for maze-family suites, 24 for bugtrap, 56 for large rooms). Early uncalibrated pilots (C1–C4) saturated (baseline success 0.925–1.00), hiding all differences; calibration is therefore treated as a precondition for any method comparison.

3 Learned Heuristic Providers

3.1 Training objectives

Three supervision schemes are used. (i) *Scalar residual regression*: predict a nonnegative detour residual on top of the baseline heuristic from local features, with a label cap; exact costs-to-go come from graph Dijkstra. (ii) *Value fields*: predict a goal-conditioned raster residual field over the map (eight input channels: occupancy, reachable-free mask, clearance, goal and start Gaussians, x, y, normalized Euclidean distance), sampled at roadmap nodes. (iii) *Space–time values*: exact backward space–time Dijkstra labels over (node, t) states for dynamic training (1,129,536 scalar samples in the heavy run). The bounded-observation study (Section 6) replaces exact labels entirely with behavior-derived returns. The choice among these objectives is itself a methodological finding: under (i) one architecture collapses to a constant at the label cap while another excels, and switching to (ii) with no other change rescues it; a properly trained (i) is later shown viable as well. Objective selection therefore precedes, and is reported separately from, any architecture or transfer claim.

3.2 Architectures and parameter matching

Providers are implemented over Multilayer Perceptrons, LSTMs (Hochreiter and Schmidhuber, 1997), ON-LSTMs (Shen et al., 2018), Hierarchical Reasoning Models (Wang et al., 2025)—including a faithful port with adaptive computation (Graves, 2016; Banino et al., 2021) verified bit-exact against the reference forward pass—U-Nets (Ronneberger et al., 2015) with FiLM conditioning (Perez et al., 2017), and product-graph graph networks. Within each comparison, arms are parameter-matched to comparable complexity and trained under one matched recipe (same data, epochs, learning-rate schedule); pooled (multi-family) training is the default, with specialist variants used only as controls.

4 Planner Integrations

The same learned estimate can enter A* in several ways; the integration is treated as an experimental factor, selected by the tightness of the admissible baseline.

Additive. The learned residual is added to the baseline heuristic, sacrificing admissibility for guidance. Used where the baseline is loose (continuous suites; path suboptimality observed 1.02–1.14).

Rank-only (focal). Search keeps an admissible band $f \leq w \cdot f_{\min}$ (Pearl and Kim, 1982; Pohl, 1970) and the learned signal only orders expansions *within* the band; at w=1.0 node optimality is untouched. Used where the baseline is tight (discrete grids), consuming the model as a ranking rather than a magnitude (Chrestien et al., 2023).

Inference-time local Bellman backup. In the bounded-observation study, learned predictions serve only as *exit values* of a radius-bounded local subgraph; one Bellman backup over that subgraph, computed at inference for the queried state, produces the heuristic (kin to a single value-iteration step (Tamar et al., 2016) at the planner interface). A scale factor α multiplies the backup; α is calibrated once on a disjoint block and then frozen.

Certified control. A separate bounded operating point runs the learned rank inside a w=1.10 focal search whose incumbent is proven against a consistent baseline bound sharing a single queue and g-state; every returned path carries a per-instance certificate. Integration lesson: an earlier variant that ran a *fresh* certifier search discarded the incumbent's work and lost all six oracle-provider comparisons; sharing the queue reversed this to six of six—so certification is implemented strictly as a shared-state search.

5 Transfer and Adaptation Protocols

5.1 Transfer distances

Three distances are distinguished: (a) unseen maps from training families; (b) unseen *families* (obstacle statistics never trained on); (c) unseen maps under restricted observation (Section 6). *Zero-shot* applies the provider frozen.

5.2 Few-shot arms

K-shot adaptation uses $K \in \{1, 2, 4, 8, 16, 32\}$ labeled target maps and three arms: rank-8 LoRA (Hu et al., 2021) (bounded and unbounded output variants), full fine-tuning, and training from scratch as a control; five adaptation seeds (three in the matched-compute rerun) on 30 fixed test maps per target. The matched-compute hardening gives every arm 10 epochs at learning rate 2×10^{-4}, and adds convolutional LoRA for the field U-Net, so capacity—not compute—is the manipulated variable; comparing bounded to unbounded adapters separates the adapter's output clamp from its rank. Dynamic-target adaptation (C9B) repeats the protocol on space–time providers (486 adapters over three backbones × aware/blind × three targets), where a single dynamic map supplies ∼25,000 labeled (node, t) states—the protocol feature that lets supervision *density*, not map count, be identified as the regime variable.

5.3 Temporal-information twins

For every dynamic backbone we train aware/blind *twins* differing in exactly one input: an 8-step future obstacle window (W=8) versus the present frame only (W=0). Twins share data, labels, recipe, and architecture, isolating the value of future-motion information both zero-shot and under adaptation.

5.4 Composition control

A zero-label alternative to adaptation interpolates 16 single-family expert adapters (four maze-density and four room-scale sources, two backbones) by task-descriptor similarity: RBF weights over descriptor axes, plus nearest, uniform, weight-space merge, and prediction-space mixture variants. Descriptor routing quality is measured separately from planning benefit so that "routing works" and "composition pays" cannot be conflated.

6 The Bounded-Observation Protocol

The hardest transfer setting restricts information access: the heuristic for a state may use only bounded local observations—a radius-0.20 subgraph around the state on a known roadmap—and may never read complete-map rasters, graph Dijkstra values, or any shortest-path supervision. Labels are *behavior-derived*: median successful returns of three fresh-start local-policy rollouts per node, with visit history reset at every labeled start. Training uses 96 suite-balanced maps (three families), validation 24, and a disjoint development block of 24 maps spanning all six families; eight outer iterations of local heuristic Bellman learning produce the checkpoint, selected on validation only.

The staged design is falsification-first: each rung of a preregistered ladder changes one factor (target statistic, calibration, analytic local constructions, training distribution, integration, scale), and only the surviving mechanism—the inference-time local backup of Section 4 with α=1.50 calibrated on 48 fresh maps—advances. Confirmation then freezes the checkpoint, radius, α, integration, gate thresholds, and analysis code *before* generating 144 zero-overlap maps (24 per family; seed offset disjoint from every prior cohort; feature caches hash-verified against live regeneration). The preregistered gate requires: 144 valid paths; pooled paired expansion CI below zero against the strongest complete-map provider; negative mean delta in at least four families; and mean/max path-cost ratios within +0.005/+0.02 of that provider. The certified w=1.10 control must separately return 144 valid paths with zero bound or certificate violations. Timing is recorded but excluded from gates (the prototype Python feature builder averages 5.14 s per map versus 0.37 s for complete-map inference; model inference itself is 0.8 ms).

7 Evaluation Metrics

Success. Fraction of episodes reaching the goal within the binding budget. Budgets differ across suites; absolute success is never compared across unrelated protocols.

Matched-solved expansion ratio. Learned-arm expansions divided by baseline expansions on maps both solve; below 1 is better. Because this conditions on joint success (and can flatter a low-success arm), it is always reported alongside—never instead of—success.

Path suboptimality. Found path cost divided by the exact optimum (graph or space–time), reported separately from search effort; additive integrations are inadmissible and their suboptimality is quantified rather than assumed away.

Timing. Feature-construction, model-inference, and search wall time recorded separately; timing never enters preregistered gates.

8 Statistical Methodology

Success contrasts use paired McNemar tests with Benjamini–Hochberg correction within stage families. Effort contrasts use bootstrap percentile intervals (10k–20k resamples) over matched maps. Because several stages evaluate repeated seeds on the same test maps, publication-grade inference clusters at the *map* level: seeds are averaged within maps before any interval, test, or permutation is computed; the paper's quoted quantities were recomputed under this grain (map-clustered bootstraps; a stratified permutation test for the halting–depth correlation, shuffling within configuration since maps are independent across depth cells). Two reporting rules are enforced throughout: success evidence and solved-only effort evidence are never merged into a single score, and negative results are stated as failures to observe an effect under the tested protocol—never as equivalence, for which no tests were run. A procedural note recorded during reanalysis: accidentally pooling additive and focal evaluation modes attenuated apparent transfer effects by roughly half, so mode filtering is part of the analysis contract, not a convenience.

9 Methodological Safeguards

Headroom gates. Before a hypothesis-bearing stage is trained, a privileged oracle must prove the benchmark can express the hypothesized effect (e.g., an exact compositional oracle beating a leg-sum baseline at expansion ratios 0.082–0.225 before the depth study; a privileged-history diagnostic proving memory relevance before the memory study; oracle integration ceilings before learned providers enter a new search). A null under a passed gate is informative; without one it is noise.

Preregistration and hard stops. From the seventh stage onward, designs, budgets, gate thresholds, and analysis grains are frozen before execution; failed development gates block confirmation rather than triggering retuning; confirmation cohorts are generated only after models, integrations, and evaluation code are frozen and hashed.

Integrity manifests. Checkpoints, feature caches, cohort manifests, raw rows, verdicts, and reports are hash-bound; caches are re-verified against live regeneration at evaluation time; duplicate evaluations must reproduce identical decision rows (wall-clock fields excluded).

Collapse forensics. Constant-prediction training collapse is detected by signature (identical final loss pinned at the label cap) and affected cells are excluded from capacity conclusions; padding manipulations are checked so pad steps cannot masquerade as useful recurrent compute. A related audit rule—any adaptive-computation "accuracy" equal to 100 minus exact-match accuracy indicates a frozen halting head—was applied to the ported architecture before its use.

Matched controls everywhere. An explicit MLP control accompanies every claim about structured architectures; twins isolate single inputs; frozen-weight reuse isolates integrations; and scratch training anchors every adaptation curve.

10 Stage-by-Stage Protocol Summary

Stage	Protocol essentials
C1–C4	9 easy suites, n=80/40 maps, budgets 100–200; pooled bases, task-LoRA experts, RBF mixtures.
C5	calibrated hard maze/dense/rooms; 160 train / ∼80 eval maps; budgets 128–168; scalar residual target with cap.
C6	goal-conditioned raster residual field (8 channels), sampled at roadmap nodes; 96/40 maps; budgets 128–168.
C7	6 suites (3 held-out families); 96/24 maps per suite; binding budgets 140/24/56; scalar+field × additive+focal matrix; exact-Dijkstra oracle.
C8	deterministic moving obstacles; space–time roadmap; aware (W=8) vs. blind (W=0) twins; 53 train / 20 eval maps per suite; 1,129,536 samples.
C9/C9H	adapt pooled bases to 3 held-out families; $K \in \{1..32\}$; LoRA(r8, bounded/unbounded), full FT, scratch; matched compute (10 epochs, LR 2×10^{-4}); 30 fixed test maps per target.
C9B	same protocol on dynamic sources (aware+blind), 486 adapters, 3 targets, $K \in \{1, 4, 16\}$.
C10	16 rank-8 experts on 2 family axes; zero-label transfer to 3 interior targets; RBF/nearest/uniform weight-merge and prediction-mix composition.
C11	compositional missions (waypoints; keys/doors), configs A/B/C, length $K \in \{0, 2, 4, 8\}$; 40 train / 25 test maps per cell; 6 arms × 3 seeds (198 checkpoints; 54,450 eval rows); exact mission oracle and leg-sum baseline; forced-compute and would-halt probes.
C12	(A) hidden slow/fast regime dynamics, memory-headroom gate, matched temporal-hierarchy pilot; (B) tied vs. untied product-graph refiner, cycles 1–8, configs A/C × $K \in \{2, 8\}$, 3 seeds, 48 checkpoints.
C13	bounded-observation heuristic (radius 0.20); behavior-derived fresh-start rollout labels (median of 3); staged falsification ladder (A–L); frozen 144-map confirmation (M); architecture substitutions (N/O).
Discrete	20×20 to 256×256 grids; Manhattan baseline; forecasting, additive residual transfer (22 suites × 3 budgets × 100 episodes), pooled multitask bases, focal ranking at $w \in \{1.0, 1.05, 1.1\}$.

Table 1: Protocol essentials by stage.

11 Compute and Reproducibility

Continuous-program stages are local single-GPU (RTX 5090) validations with one primary training seed per stage unless stated (three model/adaptation seeds where noted); the discrete program ran on a remote fleet. Code, preregistrations, per-stage raw tables, and integrity manifests are prepared for release; artifact hashing and the duplicate-evaluation contract above are what make the release checkable rather than merely available.

References

Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021.

Leah Chrestien, Tomás Pevný, Stefan Edelkamp, and Antonín Komenda. Optimize planning heuristics to rank, not to estimate cost-to-goal. *arXiv preprint arXiv:2310.19463*, 2023.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 1968. doi: 10.1109/tssc.1968.300136.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 1997. doi: 10.1162/neco.1997.9.8.1735.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

L. Kavraki, P. Svestka, J. Latombe, and M. Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Trans. Robotics Autom.*, 1996. doi: 10.1109/70.508439.

Judea Pearl and Jin H. Kim. Studies in semi-admissible heuristics. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1982. doi: 10.1109/tpami.1982.4767270.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. *arXiv preprint arXiv:1709.07871*, 2017.

Mike Phillips and Maxim Likhachev. Sipp: Safe interval path planning for dynamic environments. *2011 IEEE International Conference on Robotics and Automation*, 2011. doi: 10.1109/icra.2011.5980306.

I. Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1970. doi: 10.1016/0004-3702(70)90007-X.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *arXiv preprint arXiv:1505.04597*, 2015.

Yikang Shen, Shawn Tan, Alessandro Sordoni, and Aaron Courville. Ordered neurons: Integrating tree structures into recurrent neural networks. *arXiv preprint arXiv:1810.09536*, 2018.

David Silver. Cooperative pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 2005. doi: 10.1609/aiide.v1i1.18726.

Aviv Tamar, Yi Wu, Garrett Thomas, Sergey Levine, and Pieter Abbeel. Value iteration networks. *arXiv preprint arXiv:1602.02867*, 2016.

Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.